

Oddurboga CM5 + RP2350 — Hardware / Firmware Bring-up Report

Board: Oddurboga CM5 IO with RP2350 | Schematic rev. dated 2026-06-30 | Repo: github.com/raspipcb/oddurboga-cm5-rp2350 (branch pyqt) | Firmware FW_VERSION 1.1.2

Test session: 11–12 September 2026. Report prepared 13 September 2026.

Summary

The board was brought up from scratch: MicroPython v1.29.0 flashed to the RP2350, firmware uploaded, CM5 booted into Raspberry Pi OS (Debian trixie), PyQt app installed and started.

Five defects were found in the firmware as published. All five are of a kind that any single power-on test against real hardware would have caught immediately — the firmware did not match the schematic in three separate places, and did not run at all on MicroPython.

One unresolved hardware-level problem remains: the UART link between CM5 and RP2350 does not pass any data in either direction, although both endpoints are confirmed working and the schematic shows the wiring to be correct.

Firmware defects found and corrected

#	File	As published	Correct value	Evidence
1	all .py files	from __future__ import annotations	not supported by MicroPython	ImportError: no module named '__future__' on first import. Worked around with a __future__.py shim on the device.
2	config.py	I2C_SDA_PIN = 2 I2C_SCL_PIN = 3	I2C_SDA_PIN = 6 I2C_SCL_PIN = 7	Schematic p.4: MCU_I2C1_SDA on GPIO6, MCU_I2C1_SCL on GPIO7, with pull-ups R19/R20 (3.3K) on those two nets. I2C scan on pins 2/3 returned []; on pins 6/7 it returned 0x20 and 0x49.
3	config.py	ADS1115_ADDR = 0x48	ADS1115_ADDR = 0x49	Schematic p.5: U10 ADDR pin (1) is tied to 3V3, which selects 0x49. The note on the schematic itself says 0x48 and is also wrong.
4	config.py	BIT_RELAY0..5 = 0..5 BIT_SENSOR_24V_EN = 6	BIT_RELAY0..5 = 8..13 BIT_SENSOR_24V_EN = 0	Schematic p.6: RELAY0..RELAY5 are on GPB0..GPB5 (port B), and SENSOR_24V_EN is on GPA0. mcp23017.py maps bits 0-7 to port A and 8-15 to port B, so the published values addressed unconnected port-A pins. No relay could ever actuate.
5	pyqt/scripts /	setup-pi.sh / run.sh use pip	use apt packages	pip install fails on Debian trixie (externally-managed environment). Installed python3-pyqt5, python3-pyqt5.qtsvg, python3-serial via apt instead.

After these corrections the I2C bus enumerates correctly and all six relays actuate:

```
>>> from machine import I2C, Pin
>>> I2C(1, sda=Pin(6), scl=Pin(7), freq=100000).scan()
['0x20', '0x49'] # MCP23017 + ADS1115, as expected

>>> i2c.writeto_mem(0x20, 0x01, b'\x00') # IODIRB = outputs
>>> i2c.writeto_mem(0x20, 0x15, b'\x01') # OLATB bit0 -> relay 0 clicks
# all six relays confirmed clicking in sequence
```

Open issue: UART link CM5 ↔ RP2350 passes no data

What was tested

The PyQt app starts and opens the port, but every GET_STATUS times out:

```
2026-09-12 21:03:44 INFO link starting on /dev/ttyAMA0@115200
2026-09-12 21:03:47 WARNING !! GET_STATUS not sent (OFFLINE)
2026-09-12 21:03:51 WARNING !! GET_STATUS not sent (OFFLINE)
```

Both directions were then tested in isolation, with main.py not running (RP2350 at the REPL over USB, CM5 side at a shell):

RP2350 → CM5:

```
# CM5 (recorder started first, confirmed running in background):
$ (cat /dev/ttyAMA0 > /tmp/rx.log &)

# RP2350 (Thonny):
>>> u = UART(0, baudrate=115200, tx=Pin(0), rx=Pin(1))
>>> u.write(b'HELLO\n')
6 # 6 bytes accepted for transmission

# CM5:
$ cat /tmp/rx.log
# EMPTY - nothing received
```

CM5 → RP2350:

```
# CM5:
$ echo "PING" > /dev/ttyAMA0

# RP2350 (Thonny):
>>> print(u.read())
None # nothing received
```

What has been ruled out

- **CM5 port exists and is enabled.** dtparam=uart0=on is set in /boot/firmware/config.txt and the machine has been rebooted; /dev/ttyAMA0 is present and opens without error.
- **No console conflict.** The Linux console is on ttyAMA10 (the CM5 debug UART), not on ttyAMA0, so nothing else is holding the port.
- **RP2350 transmits.** UART(0) on GPIO0/GPIO1 accepts writes and reports bytes queued.
- **Pin assignment matches the schematic.** Schematic p.4: GPIO0 = MCU_UART0_TX, GPIO1 = MCU_UART0_RX. config.py agrees (UART_ID 0, TX 0, RX 1, 115200).
- **The crossover on the schematic is correct.** MCU_UART0_TX → R14 (51R) → CM_UART0_RX, and MCU_UART0_RX → R15 (51R) → CM_UART0_TX. TX meets RX in both directions on the drawing.
- **Firmware bugs are not the cause.** These tests were done at the REPL with main.py halted, so no application code was involved.

Remaining suspects

1. **R14 / R15 (51R) or their solder joints** — an open on either series resistor produces exactly this symptom: both endpoints healthy, nothing in between.
2. **The assembled board may differ from the schematic revision we have** (PDF dated 2026-06-30). If the UART routing or pin assignment changed after that date, our analysis is against the wrong drawing.
3. **CM5 pinmux** — whether GPIO14/15 on the CM5 are actually muxed to uart0 on this carrier, or land on a different UART instance.

Questions for Hotte

1. Was the CM5 ↔ RP2350 UART link ever verified on a physical board, and if so, how?
2. Which schematic revision does the delivered board correspond to? Is 2026-06-30 current?
3. Are R14 and R15 fitted on the delivered board, and is 51R in series correct for 115200 baud at 3.3V logic here?
4. Which CM5 UART instance is GPIO14/15 expected to appear as, and is any additional overlay required beyond `dtparam=uart0=on`?
5. Was the firmware in the repo ever executed on the RP2350? The five defects above suggest it was written against the schematic but never run.

Display status

The 7" Waveshare DSI panel (800x480) is connected to P1 (22-pin, 0.5 mm top-contact), which the schematic labels as DPHY0 of CM5. A 22-to-15-pin ribbon is required between board and panel.

`dtoverlay=vc4-kms-dsi-7inch,dsi0` has been added to `config.txt`. The panel has not yet been confirmed working; testing so far has been over HDMI.

Prepared with assistance from Claude (Anthropic). All schematic references are to `SCH_Schematic_MainBoard_2026-06-30.pdf` as published in the repository.